

Path Sums

Binary Trees, Path-Based Problems, Root-to-Leaf Sums · Mentor-Led Lab (Student Edition)

Developed by TalenciaGlobal

Difficulty ★★★ Advanced

Problem

Analyse the root-to-leaf paths of a binary tree. Report the maximum root-to-leaf path sum and count how many root-to-leaf paths sum to exactly a given target.

You are given starter code with two functions. `max_path_sum` is almost complete but has a bug, and `count_paths` is an empty stub. Fix the bug and implement the stub.

What to Compute

- Max Path Sum: the largest sum of node values along any root-to-leaf path.
- Target Paths: the number of root-to-leaf paths whose values sum to exactly the target.

Input / Output Format

Input: Line 1: the tree in level order (values and N). Line 2: the target sum.

Output: Two lines: Max Path Sum and Target Paths.

Examples

Example 1

Input: tree = 5 4 8 11 N 13 4 7 2 N N N 1, target = 22

Output:

Max Path Sum: 27

Target Paths: 1

Explanation: The four root-to-leaf paths sum to 27, 22, 26 and 18; the largest is 27 and exactly one path (5-4-11-2) equals the target 22.

Example 2

Input: tree = 1 2 3, target = 4

Output:

Max Path Sum: 4

Target Paths: 1

Explanation: Paths 1-2 (=3) and 1-3 (=4); the maximum is 4 and one path equals the target 4.

Constraints

- A path runs from the root to a leaf; sum the node values along it.
- A leaf has no children.
- Values are integers and may repeat.

Your Task

- Task 1 - Fix the bug. `max_path_sum` is flagged with a comment, but the bug's location is not given. Find the single bug so it combines children with the `MAXIMUM` (not the minimum) on the way up.
- Task 2 - Implement `count_paths` from scratch: count the root-to-leaf paths whose values sum to exactly the target.

Starter Code

Begin from the code below. One function carries a single bug (flagged by a comment, with no hint to its location); the other is an empty stub you must implement.

Python

```
import sys
from collections import deque

class Node:
    def __init__(self, v):
        self.v = v
        self.left = None
        self.right = None

def build(tokens):
    if not tokens or tokens[0] == "N":
        return None
    root = Node(int(tokens[0]))
    q = deque([root])
    i = 1
    while q and i < len(tokens):
        node = q.popleft()
        if i < len(tokens):
            if tokens[i] != "N":
                node.left = Node(int(tokens[i]))
                q.append(node.left)
            i += 1
        if i < len(tokens):
            if tokens[i] != "N":
                node.right = Node(int(tokens[i]))
                q.append(node.right)
```

```

        i += 1
    return root

# BUG: this function contains a single bug. The reference run must report the
# maximum root-to-leaf path sum, but this version does not.
def max_path_sum(node):
    if node is None:
        return None
    if node.left is None and node.right is None:
        return node.v
    best = None
    for child in (node.left, node.right):
        if child is not None:
            s = max_path_sum(child)
            if best is None or s < best:
                best = s
    return node.v + best

# TODO: implement this. It must count root-to-leaf paths whose values sum to
# exactly 'remaining' (the target passed from main). Currently a stub.
def count_paths(node, remaining):
    return 0

def main():
    data = sys.stdin.read().split("\n")
    tokens = data[0].split()
    target = int(data[1])
    root = build(tokens)
    mps = max_path_sum(root) if root else 0
    print("Max Path Sum: " + str(mps))
    print("Target Paths: " + str(count_paths(root, target)))

main()

```

C++

```

// Advanced Tree Lab 2 - Path Sums (STARTER)
#include <iostream>
#include <vector>
#include <string>
#include <queue>
#include <sstream>
#include <algorithm>
#include <set>
using namespace std;

struct Node {
    long long v;
    Node* left;
    Node* right;
    Node(long long x) : v(x), left(nullptr), right(nullptr) {}
};

Node* build(const vector<string>& t) {
    if (t.empty() || t[0] == "N") {
        return nullptr;
    }
}

```

```

Node* root = new Node(stoll(t[0]));
queue<Node*> q;
q.push(root);
size_t i = 1;
while (!q.empty() && i < t.size()) {
    Node* node = q.front();
    q.pop();
    if (i < t.size()) {
        if (t[i] != "N") {
            node->left = new Node(stoll(t[i]));
            q.push(node->left);
        }
        i++;
    }
    if (i < t.size()) {
        if (t[i] != "N") {
            node->right = new Node(stoll(t[i]));
            q.push(node->right);
        }
        i++;
    }
}
return root;
}

```

// BUG: this function contains a single bug. The reference run must report the
// maximum root-to-leaf path sum, but this version does not.

```

bool maxPathSum(Node* node, long long& result) {
    if (node == nullptr) {
        return false;
    }
    if (node->left == nullptr && node->right == nullptr) {
        result = node->v;
        return true;
    }
    long long best = 0;
    bool haveBest = false;
    long long childSum = 0;
    if (maxPathSum(node->left, childSum)) {
        if (!haveBest || childSum < best) {
            best = childSum;
            haveBest = true;
        }
    }
    if (maxPathSum(node->right, childSum)) {
        if (!haveBest || childSum < best) {
            best = childSum;
            haveBest = true;
        }
    }
    result = node->v + best;
    return true;
}

```

// TODO: implement this. It must count root-to-leaf paths whose values sum to
// exactly 'remaining' (the target passed from main). Currently a stub.

```

int countPaths(Node* node, long long remaining) {
    return 0;
}

```

```

}

int main() {
    string line1, line2;
    getline(cin, line1);
    getline(cin, line2);
    istringstream iss(line1);
    vector<string> tokens;
    string tok;
    while (iss >> tok) {
        tokens.push_back(tok);
    }
    long long target = stoll(line2);
    Node* root = build(tokens);
    long long mps = 0;
    if (root) {
        maxPathSum(root, mps);
    }
    cout << "Max Path Sum: " << mps << "\n";
    cout << "Target Paths: " << countPaths(root, target) << "\n";
    return 0;
}

```

Java

```

// Advanced Tree Lab 2 - Path Sums (STARTER)
import java.util.*;

public class Main {
    static class Node {
        long v;
        Node left, right;
        Node(long x) {
            v = x;
        }
    }

    static Node build(String[] t) {
        if (t.length == 0 || t[0].equals("N")) {
            return null;
        }
        Node root = new Node(Long.parseLong(t[0]));
        Queue<Node> q = new LinkedList<>();
        q.add(root);
        int i = 1;
        while (!q.isEmpty() && i < t.length) {
            Node node = q.poll();
            if (i < t.length) {
                if (!t[i].equals("N")) {
                    node.left = new Node(Long.parseLong(t[i]));
                    q.add(node.left);
                }
                i++;
            }
            if (i < t.length) {
                if (!t[i].equals("N")) {
                    node.right = new Node(Long.parseLong(t[i]));

```

```

        q.add(node.right);
    }
    i++;
}
}
return root;
}

// BUG: this function contains a single bug. The reference run must report the
// maximum root-to-leaf path sum, but this version does not.
static long maxPathSum(Node node) {
    if (node.left == null && node.right == null) {
        return node.v;
    }
    long best = 0;
    boolean haveBest = false;
    if (node.left != null) {
        long s = maxPathSum(node.left);
        if (!haveBest || s < best) {
            best = s;
            haveBest = true;
        }
    }
    if (node.right != null) {
        long s = maxPathSum(node.right);
        if (!haveBest || s < best) {
            best = s;
            haveBest = true;
        }
    }
    return node.v + best;
}

// TODO: implement this. It must count root-to-leaf paths whose values sum to
// exactly 'remaining' (the target passed from main). Currently a stub.
static int countPaths(Node node, long remaining) {
    return 0;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    String line1 = sc.nextLine();
    String line2 = sc.nextLine();
    String[] tokens = line1.trim().isEmpty() ? new String[0] : line1.trim().split("\\s+");
    long target = Long.parseLong(line2.trim());
    Node root = build(tokens);
    long mps = (root != null) ? maxPathSum(root) : 0;
    System.out.println("Max Path Sum: " + mps);
    System.out.println("Target Paths: " + countPaths(root, target));
}
}

```

Sample Run

Sample Input

```
5 4 8 11 N 13 4 7 2 N N N 1
```

22

Sample Output

Max Path Sum: 27

Target Paths: 1